LAPORAN TEORI
KOMUNIKASI DAN JARINGAN KOMPUTER



DANAR IKRAM TIRTA KUSUMA

1226800009

PROGRAM STUDI S2 TEKNIK INFORMATIKA DAN KOMPUTER

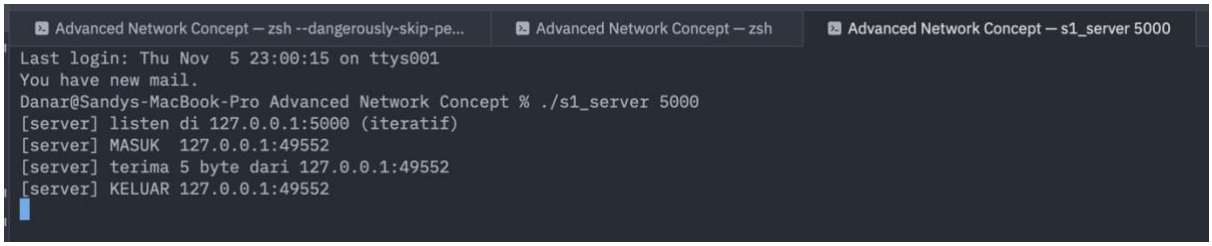
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

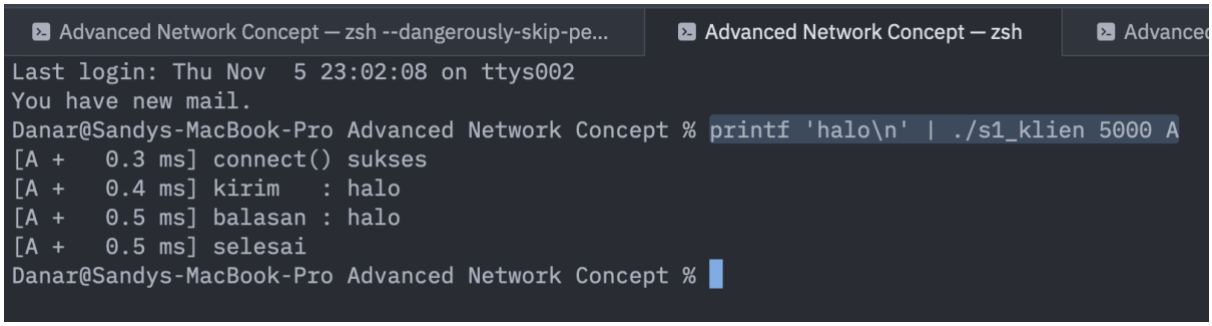
2026

- **S1 Server Iteratif**

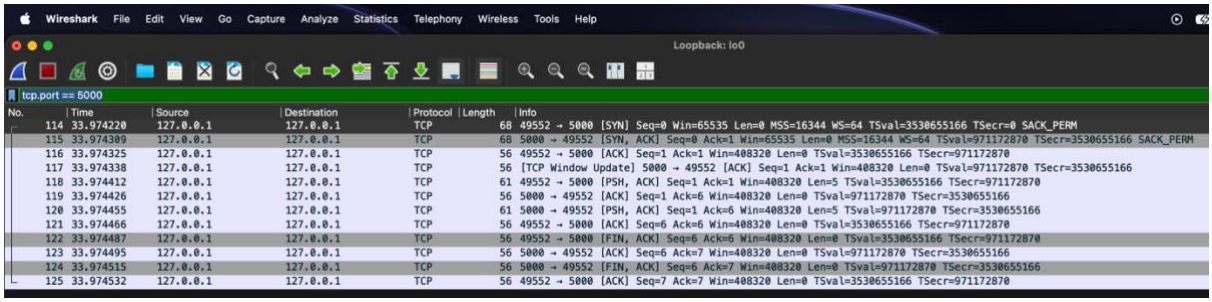
Menjalankan code s1_server:



Menjalankan perintah printf 'halo\n' | ./s1_klien 5000 A:



Wireshark capture dengan filter tcp.port == 5000:



Analisis:

A. TCP 3-Way Handshake (Pembentukan Koneksi)
   - Paket 1–3: SYN → SYN, ACK → ACK.
   - Dipicu saat s1_klien.c:37 memanggil connect().
   - TCP stack di kernel server merespons berdasarkan socket listener yang sudah disiapkan oleh listen(ls, 16) di s1_server_iteratif.c:34.
   - Setelah handshake selesai di level OS, barulah fungsi accept() di server melepas blokirannya (unblocks) dan mengembalikan descriptor cs.

B. Pertukaran Data (Echo Transmission)
   - Klien mengirim paket dengan flag [PSH, ACK]; payload 5 byte (h, a, l, o, \n).
   - Server membalas paket [PSH, ACK] yang berisi 5 byte yang sama.
   - Klien menjalankan send(s, baris, len, 0) di baris 44.
   - Server membaca dengan recv(cs, buf, ...) di baris 59, lalu memantulkannya dengan send(cs, buf, n, 0) di baris 62.
   - Flag PSH (Push) memberi sinyal agar TCP stack segera meneruskan byte yang diterima ke buffer aplikasi tanpa menunggu buffer segmen penuh.

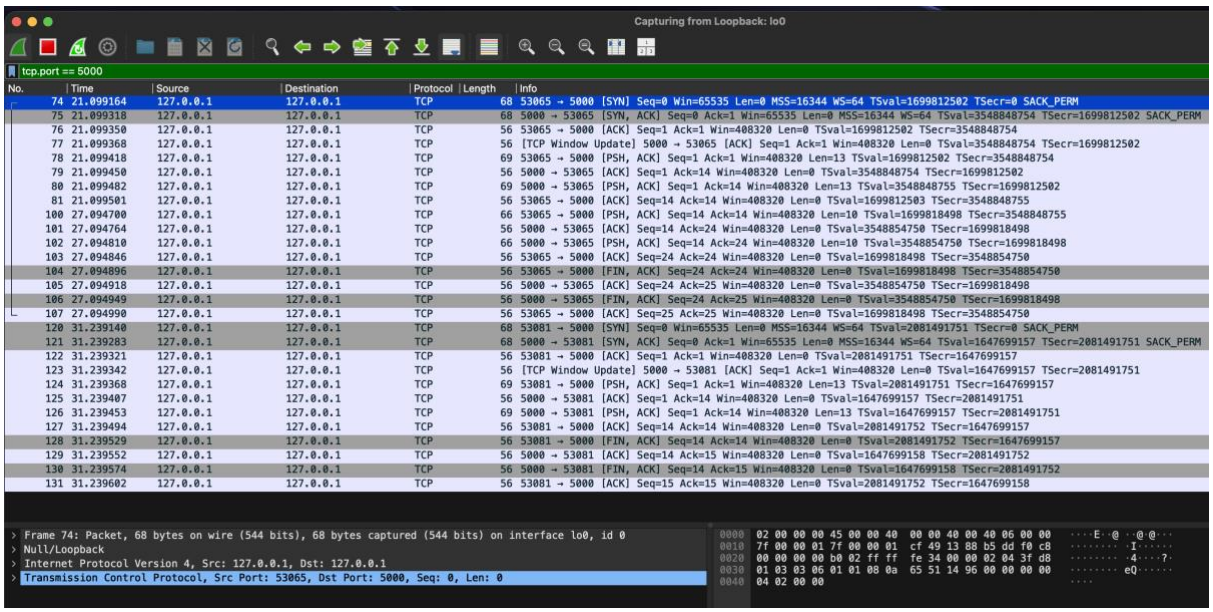
C. Connection Teardown (Penutupan Koneksi 4-Way)
   - Paket Penutupan: FIN, ACK → ACK → FIN, ACK → ACK.
   - Klien selesai membaca stdin dan memanggil close(s) di s1_klien.c:56. Ini mengirim flag FIN.
   - Server mendeteksi penutupan saat recv() mengembalikan 0 (EOF). Server kemudian mencatat KELUAR dan mengeksekusi close(cs) di baris 69, memicu FIN balasan dari server.

Uji coba Bottleneck Server Iteratif:

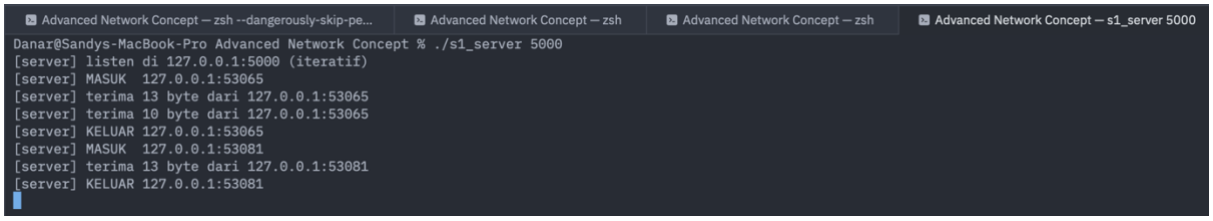
1. Wireshark capture di lo0 dengan filter tcp.port == 5000.
2. Menjalankan Server di Terminal 1: ./s1_server 5000
3. Di Terminal 2, menjalanka Klien A dengan jeda waktu:
   (echo "Pesan dari A"; sleep 6; echo "Selesai A") | ./s1_klien 5000 KlienA
4. Di Terminal 3, segera jalankan Klien B (selagi Klien A masih berjalan):
   printf 'Pesan dari B\n' | ./s1_klien 5000 KlienB

Hasil:

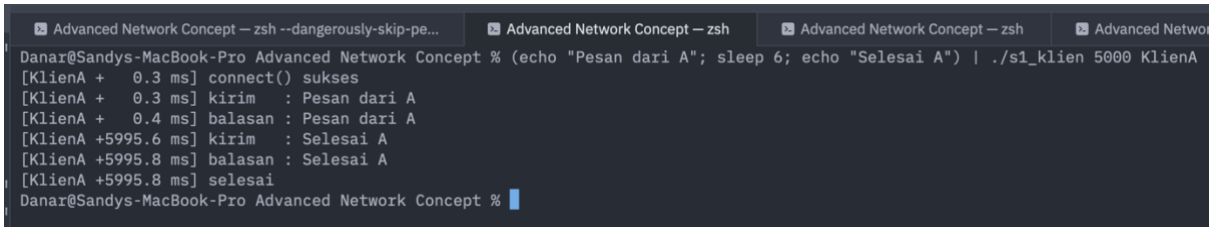
Capture wireshark:



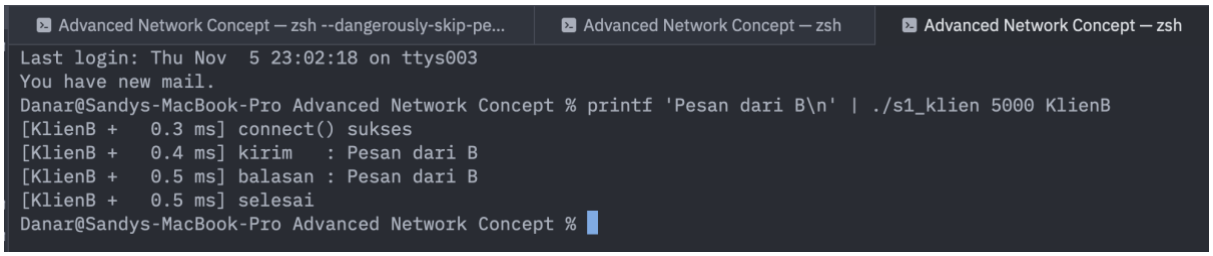
Output terminal 1:



Output terminal 2:



Output terminal 3:



Analisis:

A. Di level OS/Kernel:
   - Paket SYN dari Klien B tetap di-ACK oleh kernel server (SYN, ACK), dan handshake Klien B selesai. Ini membuktikan bahwa TCP 3-way handshake ditangani oleh kernel OS dan disimpan di antrean backlog (listen(..., 16)).

B. Di level Aplikasi:
   - Klien B sudah mengirim data (PSH, ACK), dan kernel server mengirim ACK TCP.
   - Tetapi tidak ada paket echo balasan dari Server ke Klien B selama ~6 detik. Server sama sekali tidak memproses pesan Klien B karena thread/process server masih tertahan (blocked) di dalam loop while (recv) milik Klien A.
   - Begitu Klien A selesai dan mengirim FIN, barulah server memanggil accept() untuk Klien B dan membalas echo Klien B.

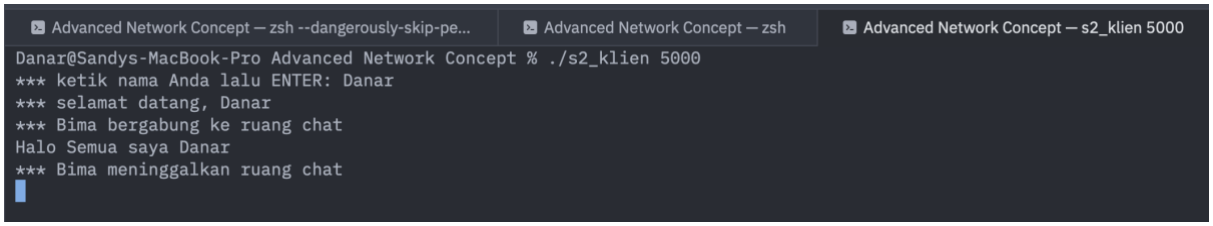
- **S2 Server Chat Multi-Pengguna dengan poll()**

  Uji Fitur Interaktif & Broadcast:

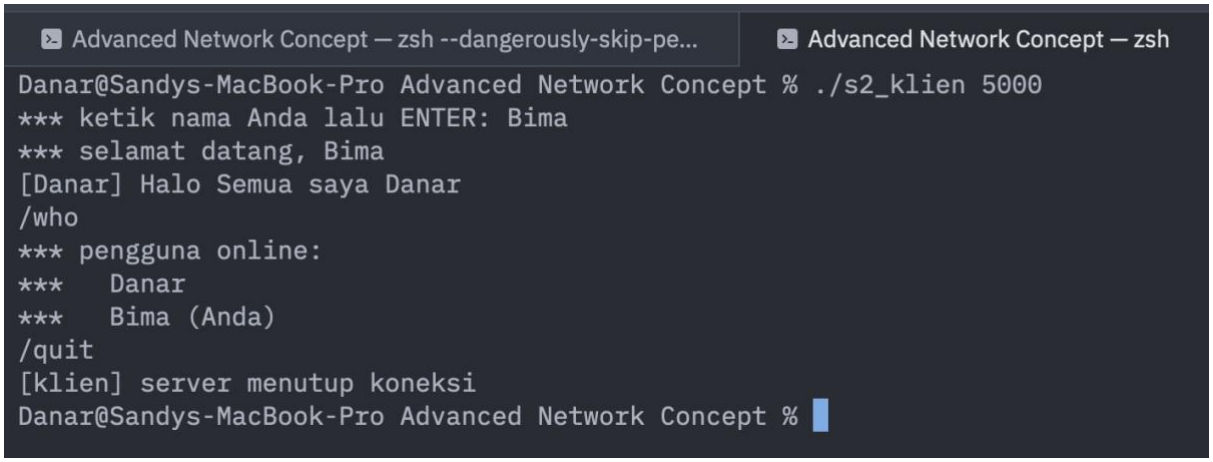
  Terminal 1 - Server:



  Terminal 2 - Klien 1 (Misal: User Danar):



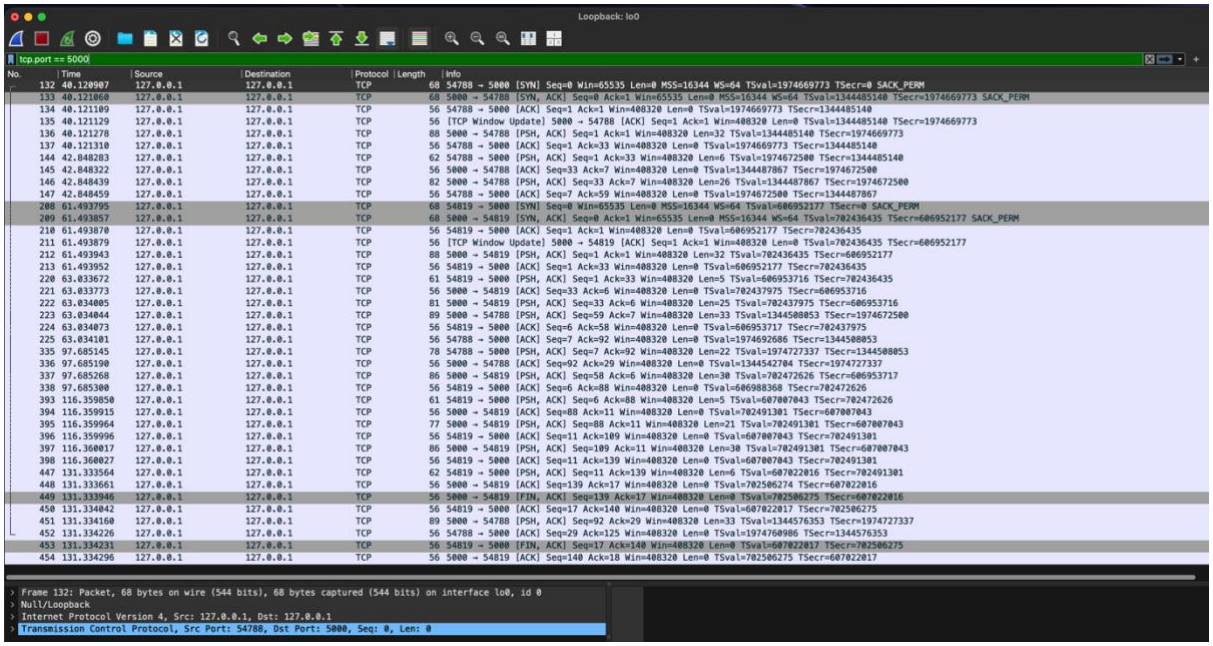
  Terminal 3 - Klien 2 (Misal: User Bima):



  Analisis:

  1. Di Terminal 2 (Danar), ketik: halo semua, saya Danar! Di Terminal 3 (Bima), akan langsung muncul: [Danar] halo semua, saya Danar!
  2. Di Terminal 3 (Bima), coba perintah status /who → Server mengembalikan daftar pengguna aktif: Danar dan Bima.
  3. Di salah satu klien, ketik /quit. Maka Klien lain akan menerima notifikasi: *** <nama> meninggalkan ruang chat.

Hasil capture wireshark:



Analisis Perbandingan: Tahap 1 (s1) vs Tahap 2 (s2):

Aspek Karakteristik	Tahap 1 (s1_server_iteratif & s1_klien)	Tahap 2 (s2_server_poll & s2_klien)
Model Eksekusi Server	Iteratif (Sekuensial): Melayani 1 klien hingga tuntas baru melayani klien berikutnya.	Event-Driven / I/O Multiplexing (poll): Satu proses & thread dapat melayani hingga 32 klien simultan.
Panggilan I/O	Blocking murni: accept() memblokir loop, lalu recv() memblokir seluruh server.	Non-blocking loop dengan event polling: poll() hanya memicu recv() atau accept() saat data sudah siap di buffer OS.
Pola Komunikasi	Point-to-Point Echo (Request -> Reply ke pengirim saja).	Multi-user Chatroom & Broadcast (Pesan disiarkan ke semua klien aktif).
Penanganan TCP Stream	Naif (Langsung tulis): Mengasumsikan 1 recv() berisi 1 pesan utuh.	Framing / Perakitan Baris (rakit_baris): Menampung potongan paket TCP stream dan memisahkan berdasarkan baris \n.
Perilaku Sisi Klien	Klien Blocking: fgets() memblokir pembacaan socket.	Klien Full-Duplex Multiplexed: poll() membaca keyboard dan socket secara paralel.
Status (Statefulness)	Stateless (Server tidak mencatat identitas klien).	Stateful Session (Validasi nama, status registrasi, perintah internal /who dan /quit).
Ketahanan Concurrency	Sangat Buruk (Vulnerable to DoS): Klien yang sengaja diam (idle) akan membuat seluruh klien lain terblokir di antrean TCP backlog.	Sangat Baik: Klien yang sedang idle tidak memakan CPU dan tidak memblokir klien lain yang aktif bertukar pesan.

LINK GITHUB:
https://github.com/DelTdev/komunikasi-jaringan_assignment